

Algoritmi e Strutture Dati – 2/7/2024 – Parte A

A1 – Complessità – Punti ≥ 8

Trovare un limite superiore ed un limite inferiore, i più stretti possibili, per la seguente equazione di ricorrenza, utilizzando il *metodo di sostituzione*.

$$T(n) = \begin{cases} 1 & n < 4 \\ 7T(\lfloor n/4 \rfloor) + n\sqrt{n} & n \geq 4 \end{cases}$$

A2 – Più o meno – Punti ≥ 10

Scrivere un algoritmo

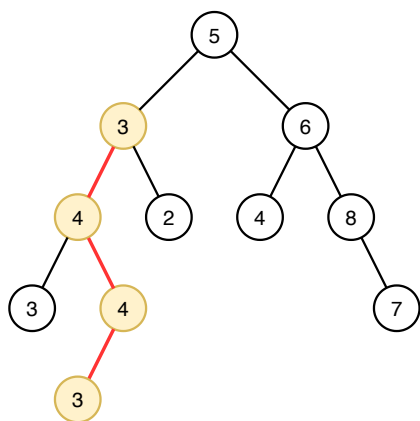
int longestPlusMinus(TREE *T*, **int** *k*)

che prenda in input un albero binario contenente numeri interi (accessibili tramite l'attributo *key* dei nodi) e restituisca la lunghezza del cammino nodo-foglia *più-o-meno-k* più lungo contenuto nell'albero.

Un cammino nodo-foglia parte da un qualunque nodo *t* e raggiunge uno qualsiasi dei nodi foglia contenuti nel sottoalbero radicato in *t*. Tale cammino è detto *più-o-meno-k* se il valore contenuto in ogni nodo del cammino differisce dal valore contenuto nel padre di tale nodo per al più *k*.

Discutere informalmente la correttezza della soluzione proposta e calcolare la complessità computazionale.

Nell'albero seguente e con $k = 1$, il più lungo cammino nodo-foglia evidenziato è quello contenuto nell'albero e l'algoritmo deve restituire 3.



A3 – Circuiti matriciali – Punti ≥ 12

State sviluppando un generatore casuale di mappe per un videogioco di corse automobilistiche. Ogni mappa è rappresentata da una matrice booleana *M* di dimensione $n \times n$, dove le celle con valore **true** rappresentano ostacoli e non possono essere attraversate, mentre le celle con valore **false** sono libere e attraversabili. Ogni auto può muoversi in quattro direzioni: sinistra, destra, alto e basso. Le auto non possono muoversi in diagonale o in altre direzioni non ortogonali.

Il vostro compito è scrivere un algoritmo

boolean hasCircuit(**boolean**[][] *M*, **int** *n*)

che prenda in input la matrice e verifichi se all'interno di essa esiste un circuito, ovvero un percorso per cui sia possibile partire da una cella, muoversi tra le celle in una delle quattro direzioni consentite, e eventualmente tornare alla cella di partenza senza attraversare celle già percorse.

Discutere informalmente la correttezza dell'algoritmo e la sua complessità computazionale.

Per esempio, nella mappa di esempio a sinistra, esiste un circuito; nella mappa di destra, no.

